

Lab 10 — Persistence: NVS — Data That Survives Power Loss

ESP32-S3 FIRMWARE LAB SERIES · PHASE 2 · PAST THE COURSEWORK

Estimated time	1–2 hrs
Introduces	flash partitions, namespaces, the commit step
Prerequisites	Lab 0.
Beyond the board	Nothing. The flash-size setting must be correct for the part fitted.
Target	ESP32-S3 (written against an N16R8 module: 16 MB flash, 8 MB octal PSRAM)
Framework	ESP-IDF v5.5, CMake and Ninja, driven by <code>idf.py</code>

NOTE

How to read this lab

Sections run **§10.1 objective**, **§10.2 parts and wiring**, **§10.3 how it works**, then the code, a line-by-line walkthrough, the documentation map, and verification. Callout boxes are tagged **[WARNING]**, **[KEY POINT]**, **[NOTE]**, **[DOCS]** and **[TM4C BRIDGE]**. Figures are given as captioned text and tables so every value in them is searchable and checkable.

The **TM4C bridge** notes map each topic onto the equivalent bare-metal, register-level work on a TI TM4C123 (Cortex-M4F). Skip them if that is not your background; nothing later depends on them.

10.1 Objective & TM4C bridge

Store a value that survives a reboot and a full power cycle. Every device that remembers a Wi-Fi password, a calibration constant, or a serial number does this, and on the ESP32 it is also a prerequisite for Wi-Fi and BLE, which keep their own calibration data in the same store.

TM4C BRIDGE

The TM4C had EEPROM that you wrote through its registers for exactly this purpose. NVS is that idea with a key-value API and flash wear handling layered on top. The underlying constraint is the same: writing non-volatile memory is slow, has limited endurance, and needs care.

10.2 Parts & wiring

Item	Status
ESP32-S3 board	nothing external — the flash is on the module

WARNING**Confirm your flash size before this lab**

This is the first lab where a wrong flash-size setting can actually bite. If `sdkconfig` still says 2 MB while your part is 16 MB, partition behaviour becomes unpredictable and the failure looks like data corruption rather than a config error. `idf.py menuconfig` → Serial flasher config → Flash size → **16 MB**.

10.3 How it works

Everything you have written so far forgets everything the moment power is removed. This lab is about making one number survive. Assume nothing about flash memory; it is all below.

Volatile and non-volatile

Memory splits into two families, and the word that separates them is **volatile** — used here in its ordinary English sense of "evaporates," which is a different thing from the C keyword you met in Lab 4.

Volatile memory (RAM) needs continuous power to hold its contents. Cut power and it is blank. Every variable, every buffer, your entire stack, all of it lives here. **Non-volatile memory** (flash) keeps its contents with no power at all. Your program image lives here, which is why the chip still runs the same firmware after you unplug it overnight.

So the question this lab answers is simply: how do I put a value I computed at runtime into the non-volatile half?

Why you cannot just write to flash the way you write to RAM

Why you cannot just write a variable to flash

RAM — byte addressable



Change one byte: just write it.
Unlimited writes. Instant.

But it is gone at power-off.

FLASH — erased in whole blocks



To change ONE byte you must erase this whole block first, then rewrite all of it.

Slow, and blocks wear out (~100,000 erases).

That is what NVS is for. It hides all three problems:

- It appends new values to free space instead of erasing in place, so a small change is fast.
- It spreads writes across the partition (wear levelling) so no one block dies early.
- It keeps the old copy readable until the new one is complete, so power loss mid-write is survivable.

The price you pay for that:

- You address data by a STRING KEY inside a NAMESPACE, not by a memory address.
- Writes are buffered in RAM and only reach flash when you call `nvs_commit()`. This is the line everyone forgets.

Figure 10.1. Flash cannot be modified a byte at a time. Changing one value means erasing an entire block and rewriting it, blocks wear out after a finite number of erases, and a power cut mid-write can destroy data. NVS exists to handle all three.

Why you cannot simply write a variable to flash.

RAM, byte addressable: to change one byte you just write it. Unlimited writes, instant. But it is gone at power-off.

FLASH, erased in whole blocks: to change ONE byte you must erase an entire block first (typically 4 KB), then rewrite all of it. Slow, and blocks wear out after roughly 100,000 erases.

That is what NVS is for. It hides all three problems: - It appends new values to free space instead of erasing in place, so a small change is fast. - It spreads writes across the partition (wear levelling) so no one block dies early. - It keeps the old copy readable until the new one is complete, so power loss mid-write is survivable.

The price you pay: - You address data by a STRING KEY inside a NAMESPACE, not by a memory address. - Writes are buffered in RAM and only reach flash when you call `nvs_commit()`. This is the line everyone forgets.

Three physical facts about flash memory drive the whole design, and they are worth understanding rather than accepting.

1. You can clear bits but not set them. Programming flash can only change a 1 to a 0. To turn a 0 back into a 1 you must **erase**, and erasing works on a whole **block** at a time — typically 4 KB on this part, not one byte. So "change this one integer" really means "read out 4 KB, erase 4 KB, write back 4 KB with one number different." That is slow, on the order of tens of milliseconds.

2. Erase cycles are finite. Each erase physically degrades the oxide layer in the cells. Datasheets guarantee something like 100,000 erase cycles per block. That sounds enormous until you write a counter once a second: 100,000 seconds is about a day. Hammering one location will genuinely destroy it.

3. Power can fail mid-write. If the supply drops during that erase-and-rewrite cycle, the block is left half-erased, and whatever else was sharing those 4 KB is destroyed along with your value.

What NVS actually does about it

NVS (Non-Volatile Storage) is a small key-value database that lives in its own flash partition and solves all three problems for you.

Problem	What NVS does
Erase-before-write is slow	Writes are appended to free space rather than modifying in place. The old entry is marked stale and the new one becomes current, so a small update does not trigger an erase at all until a page fills.
Blocks wear out	Wear levelling: because writes append and pages are reclaimed in rotation, the writes are spread across the whole partition instead of hammering one spot.
Power loss corrupts	The previous value stays valid and readable until the new one is fully written. An interrupted write leaves you with the old value, not with garbage.

This is the same class of machinery that makes an SSD work. You are getting a miniature version of it, built into the SDK.

What it costs you

The abstraction is not free, and both costs show up directly in the code.

You address data by name, not by address. There is no pointer to your stored integer. You store it under a string **key** (`"boots"`) inside a **namespace** (`"storage"`), and you get it back by asking for that key. A namespace is just a labelled drawer that keeps your keys from colliding with another module's — Wi-Fi stores its calibration data in NVS too, under its own namespace, so your key called `config` and its key called `config` coexist without either knowing.

Writes are buffered until you commit. NVS batches changes in RAM for efficiency and **nothing reaches flash until `nvs_commit()`**. This is the single most common NVS bug: the code runs, no error appears, and the value silently fails to persist.

Where NVS sits in the flash

The 16 MB of flash is divided by a **partition table** written near the start of the chip. One region holds the bootloader, one or more hold application images, and one is labelled `nvs`. Keeping NVS in its own partition is what lets you reflash your application without wiping stored settings — the flash tool only overwrites the app region.

You will meet this table again in Lab 15, where two application partitions make over-the-air updates possible. It is worth opening now: run `idf.py partition-table` and look at the `nvs` entry and its size.

WARNING

Confirm your flash size before this lab

This is the first lab where a wrong flash-size setting can genuinely misbehave, because partition offsets are computed against the configured size. If `sdkconfig` still says 2 MB while your part is 16 MB, the symptom looks like data corruption rather than a configuration error. `idf.py menuconfig` → Serial flasher config → Flash size → **16 MB**.

Three tiers of persistence — picking the right one

By the end of Lab 12 you will have three places to put data, and choosing correctly is a real engineering decision.

Where	Survives	Speed	Wears out?	Use for
Ordinary RAM	Nothing	Instant	No	Working data
RTC RAM (Lab 12)	Deep sleep, not power loss	Instant	No	Wake counters, cycle state
NVS (flash)	Full power loss	Slow writes	Yes	Settings, calibration, serial numbers

The wear column is the one people forget. Writing to NVS in a fast loop is a real way to destroy hardware. If you need to track something that changes constantly, keep it in RAM and write a summary to NVS occasionally — which is exactly the exercise in §12.7.

10.4 Procedure & code

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "nvs_flash.h"
#include "nvs.h"
#include "esp_log.h"

void app_main(void) {
    esp_err_t err = nvs_flash_init();
    if (err == ESP_ERR_NVS_NO_FREE_PAGES || err == ESP_ERR_NVS_NEW_VERSION_FOUND) {
        ESP_ERROR_CHECK(nvs_flash_erase()); // standard boot pattern: wipe & retry if layout
        ESP_ERROR_CHECK(nvs_flash_init());
    }

    nvs_handle_t h;
    ESP_ERROR_CHECK(nvs_open("storage", NVS_READWRITE, &h)); // "storage" = a namespace

    int32_t boots = 0;
    nvs_get_i32(h, "boots", &boots); // NOT_FOUND on first boot -> boots stays 0
    boots++;
    ESP_ERROR_CHECK(nvs_set_i32(h, "boots", boots));
    ESP_ERROR_CHECK(nvs_commit(h)); // nothing persists until THIS line
    nvs_close(h);

    ESP_LOGI("lab10", "this device has booted %ld times", boots);
    vTaskDelay(pdMS_TO_TICKS(1000));
    esp_restart(); // reboot so you can watch it climb
}
```

10.5 Code walkthrough, line by line

Twenty-seven lines, and one of them is the line everyone forgets.

Lines 3-4 · `#include "nvs_flash.h" / #include "nvs.h"`

Two headers, and the split reflects two layers. `nvs_flash.h` is the **partition-level** API: mounting, erasing, initialising the whole store. `nvs.h` is the **data-level** API: opening a namespace, getting and setting keys. You almost always need both.

Line 8 · `esp_err_t err = nvs_flash_init();`

Mounts the NVS partition and reads its page headers. Note the return value is **captured into a variable** rather than wrapped in `ESP_ERROR_CHECK`, which is deliberate — the next line needs to inspect it, because two specific failures are recoverable.

Line 9 · `if (err == ESP_ERR_NVS_NO_FREE_PAGES || err == ESP_ERR_NVS_NEW_VERSION_FOUND)`

Two real-world conditions worth naming.

NO_FREE_PAGES means the partition is full of stale entries with nowhere left to append. Remember from §10.3 that NVS appends rather than overwrites; eventually it must reclaim space, and if it cannot, this is the error.

NEW_VERSION_FOUND means the data was written by a newer NVS format than this firmware understands — which happens when you downgrade firmware on a device that has already run a newer build.

`||` is logical OR, so either condition triggers the recovery.

Lines 10-11 · `nvs_flash_erase(); nvs_flash_init();`

Wipe the whole partition and try again. This block is **boilerplate you will copy into the top of nearly every ESP-IDF application**, because Wi-Fi and Bluetooth both refuse to start without a mounted NVS — which is exactly why this lab comes before Lab 13.

WARNING

Notice that this is destructive

`nvs_flash_erase()` throws away every stored setting on the device. In a lab that is fine. In a product it would silently delete the user's Wi-Fi credentials and your factory calibration, so real firmware thinks harder here — typically logging the event, and distinguishing "corrupt, must wipe" from "full, should compact." Copying this boilerplate without noticing what it does is a genuine trap.

Line 14 · `nvs_handle_t h;`

A handle, the same opaque-token pattern as every driver in this series. It identifies an open namespace, and it must be closed when you are done (line 22) because each open one consumes a slot in a fixed-size table.

Line 15 · `nvs_open("storage", NVS_READWRITE, &h);`

Opens the namespace named `"storage"` for reading and writing, writing the handle into `h`. Three things worth noting.

The namespace name is **yours to choose** and is created on first use — there is no separate "create namespace" call. Keep it under 15 characters; NVS enforces a limit.

NVS_READWRITE versus **NVS_READONLY** is worth using deliberately. Opening read-only when you only intend to read makes the intent explicit and means a stray write cannot compile-and-corrupt. It also does not create the namespace if it does not exist, so a read-only open of a fresh device returns **ESP_ERR_NVS_NOT_FOUND** — occasionally surprising.

`&h` is the address-of operator: the function writes the handle back into your variable, the standard output-parameter pattern.

Line 17 · `int32_t boots = 0;`

The default is set first, and that ordering is load-bearing. On a factory-fresh device the key does not exist yet, and the read on the next line will leave this variable untouched. Because it already holds

0, the program does exactly the right thing on first boot with no special-case code. Declare it uninitialised and first boot would increment whatever garbage was on the stack.

`int32_t` rather than `int` because NVS has separate typed calls per width and you must match them exactly.

Line 18 · `nvs_get_i32(h, "boots", &boots);`

Reads the stored value into `boots`. The `i32` suffix names the type: NVS provides `nvs_get_i8`, `u8`, `i16`, `u16`, `i32`, `u32`, `i64`, `u64`, plus `str` for strings and `blob` for arbitrary bytes. **The type you write must match the type you read** or you get `ESP_ERR_NVS_TYPE_MISMATCH`.

This is the one call deliberately not wrapped in `ESP_ERROR_CHECK`, and understanding why matters. On first boot it returns `ESP_ERR_NVS_NOT_FOUND`, which is a completely *expected* outcome, not a failure. Wrapping it would abort the program on every fresh device. Knowing which errors are exceptional and which are normal control flow is a real part of using an API well.

Line 19 · `boots++;`

The modify step of read-modify-write. Nothing subtle, but note the whole pattern — read into a default, modify in RAM, write back — is the standard shape of every settings update you will ever write.

Line 20 · `nvs_set_i32(h, "boots", boots);`

Stages the new value. Despite the name, this does not write to flash. It records the change in the driver's RAM buffer and returns.

Line 21 · `nvs_commit(h);`

The line everyone forgets, exactly once. This is what actually flushes buffered changes into flash. Omit it and the program runs perfectly, reports no error, and the counter resets to 1 on every single boot — a maddening symptom with no diagnostic pointing at the cause.

The buffering exists for a good reason: it lets you set twenty keys and pay the flash-write cost once, which matters given everything in §10.3 about erase cost and wear.

You have now seen this stage-then-apply pattern three times: `led_strip_refresh` in Lab 1, `ledc_update_duty` in Lab 6, and `nvs_commit` here. Once you notice it as a pattern, it stops catching you out.

Line 22 · `nvs_close(h);`

Releases the handle. Handles come from a fixed-size pool, so a program that opens without closing in a loop will eventually fail to open anything.

Line 24 · `ESP_LOGI("lab10", "this device has booted %ld times", boots);`

`%ld` is the conversion for a **signed long**, matching `int32_t`. Using `%d` here happens to work on this architecture, which is worse than failing, because the habit breaks the moment you compile for a platform where `long` is wider.

Lines 25-26 · `vTaskDelay(...); esp_restart();`

The delay gives the log line time to flush over serial before the chip reboots — without it you can lose the last message. `esp_restart()` then reboots so you can watch the count climb 1, 2, 3 without touching anything.

Note this is a **soft reset**: the chip restarts but power is never removed. The stronger test is to unplug the board entirely and plug it back in. A value that survives that is genuinely in flash, and it also proves you are not accidentally observing some RAM that happened to persist across a warm reboot.

10.6 Docs & references

DOCS

Trace every line to its source

`nvs_flash_init`, `nvs_open`, `nvs_get_i32`, `nvs_set_i32`, `nvs_commit`, `nvs_close` — API Reference → Storage → **Non-Volatile Storage**. Read the section on the commit behaviour and the list of typed getters and setters.

Error codes — the same page lists `ESP_ERR_NVS_*`. Knowing that `NOT_FOUND` is normal on first boot is the kind of detail that separates working code from code that aborts on a fresh device.

Partition tables — API Guides → **Partition Tables**. Look at the default table and find the `nvs` entry and its size. You will return here for Lab 15.

Flash endurance — the flash chip's own datasheet states the guaranteed erase cycles. Worth seeing the real number.

10.7 Verify, troubleshoot, your turn

Verify. The count increases by one on every reboot: 1, 2, 3. Then power the board down completely, wait, and reconnect — the count continues from where it was rather than restarting at 1.

Troubleshoot

Symptom	Cause and fix
Count always reads 1	Missing <code>nvs_commit</code> . The overwhelmingly likely cause.
<code>nvs_open</code> fails	NVS was not initialised, or the partition table has no <code>nvs</code> entry.
Init returns <code>NO_FREE_PAGES</code> every boot	The partition is too small, or the flash-size setting is wrong.
Value reads back as nonsense	Type mismatch — written with <code>set_i32</code> and read with a different width.
Works until you reflash, then resets	Expected if you erased flash. <code>idf.py flash</code> alone preserves NVS; <code>erase-flash</code> does not.

Your turn

1. **Store a string.** Use `nvs_set_str` and `nvs_get_str` to save a device name. Reading strings is a two-call pattern — ask for the length first, then read into a buffer you sized from it — and working that out from the documentation is exactly the skill this series is built on.

2. **Store your Lab 5 calibration.** Save the resting accelerometer offset so the device self-calibrates once and remembers. This is a genuinely useful feature rather than a toy, and it is the kind of thing that makes a project look finished.
3. **Pull the power mid-write** deliberately, in a loop that writes constantly, then reboot and see whether the store survives. NVS is designed to be robust to this; confirming it yourself is a good demonstration of why a real key-value store beats writing raw flash.

Errata and corrections

Defects confirmed against the hardware and the vendor documentation, with the evidence that established each one. They are printed rather than quietly patched because how an error was caught is usually more instructive than the error.

Additional corrections

Missing include. The listing calls `esp_restart()` without `#include "esp_system.h"`. Add it.

Bench realities

Problems that stop this lab but are not about the peripheral it teaches. Each one cost real time on a real bench.

The flash-size setting, one more time

This has now caused problems in Labs 10, 15 and 23 and it is worth doing once, permanently: `idf.py menuconfig` → Serial flasher config → Flash size → **16 MB**. Then confirm rather than assume:

```
grep ESPT00LPY_FLASHSIZE build/config/sdkconfig.h
```

Better still, put it in `sdkconfig.defaults` at the project root so every new project inherits it:

```
CONFIG_ESPT00LPY_FLASHSIZE_16MB=y
CONFIG_ESPT00LPY_FLASHSIZE="16MB"
CONFIG_SPIRAM=y
CONFIG_SPIRAM_MODE_OCT=y
```

The series at a glance

Each lab is a complete, independently buildable ESP-IDF project. This is Lab 10.

#	Lab	Phase
00	Toolchain: hello, ESP32-S3	Foundations
01	GPIO: RGB LED and BOOT button	Foundations
02	FreeRTOS tasks and drift-free timing	Foundations
03	UART and the loopback test	Foundations
04	Hardware timers and the 1 kHz problem	Foundations
05	I ² C and the IMU bring-up	Foundations
06	PWM with the LEDC peripheral	Foundations
07	The sensor pipeline (capstone)	Foundations
08	GPIO interrupts and ISR-safe code	Past the coursework
09	SPI, full-duplex and fast	Past the coursework
10 ←	NVS: data that survives power loss	Past the coursework
11	Unit testing with Unity	Past the coursework
12	Deep sleep and waking up	Past the coursework
13	Wi-Fi station and JSON telemetry	Past the coursework
14	BLE GATT with NimBLE	Past the coursework
15	OTA and the rollback safety net	Past the coursework
16	Priority inversion and mutexes	Past the coursework
17	CAN (TWAI): broadcast by message ID	Past the coursework
18	PID, anti-windup and derivative kick	Past the coursework
19	The ML inference harness	Gap-closers
20	Continuous ADC: DMA-backed acquisition	Gap-closers
21	On-chip debugging: JTAG, OpenOCD, GDB	Gap-closers
22	A real display (SPI TFT + DMA)	Gap-closers
23	Production test mode	Gap-closers

Written against ESP-IDF v5.5 on an ESP32-S3 N16R8 module. Pin assignments are choices, not constraints — the ESP32-S3 GPIO matrix routes almost any peripheral to almost any pin, with the exception of GPIO26–GPIO37, which are committed to the on-board flash and PSRAM and must never be used.